



STUDENT DETAILS

Name: Job Roman Barasa

Admission Number: BSCCS/2024/42922

Course: Bachelor of Science in Computer Science

Unit Code: BIT 4138

Unit Name: Advanced Cryptography

Lecturer Name: Mr Michael Nyoro

Academic Year: 4.2

PROJECT DETAILS

Project Title: Course Work Advanced Cryptography

Creative Tasks Done

Encrypted and decrypted words using Vigenere and Caesar Cipher

Created random number generators

Modified an already existing XOR cipher

Additional assignments week 1 to 3

Selected Technologies

Python

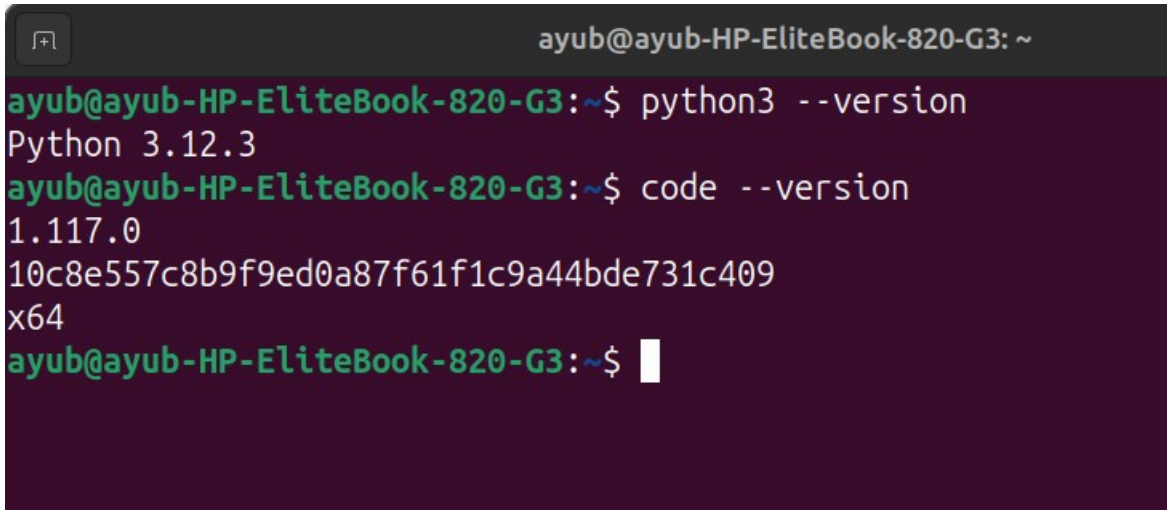
Python libraries

Github Link: [Files](#)

Week 1: Cryptography Environment Setup.....	3
Installation and Configuration of Cryptography Development Environment.....	3
Reflection.....	4
Class Activity 1: Manual Encryption.....	5
Class Activity 2: Cipher Comparison Table.....	6
Practical Task Report: Comparison of Caesar Cipher and Vigenère Cipher.....	7
Extra Practical Task.....	8
Week 2: Classical Cryptography and Cipher Design.....	9
Implementation of Classical Encryption Algorithms.....	9
Reflection.....	11
Practical Demonstrations.....	12
Class Activities.....	14
Advanced Practice Tasks.....	16
Week 3: Stream Ciphers and Randomness Testing.....	18
Implementation of Stream Cipher Systems and Random Sequence Analysis..	18
Reflection.....	19
Practical Demonstrations.....	20
Assignment 1.....	22
Assignment 2.....	24

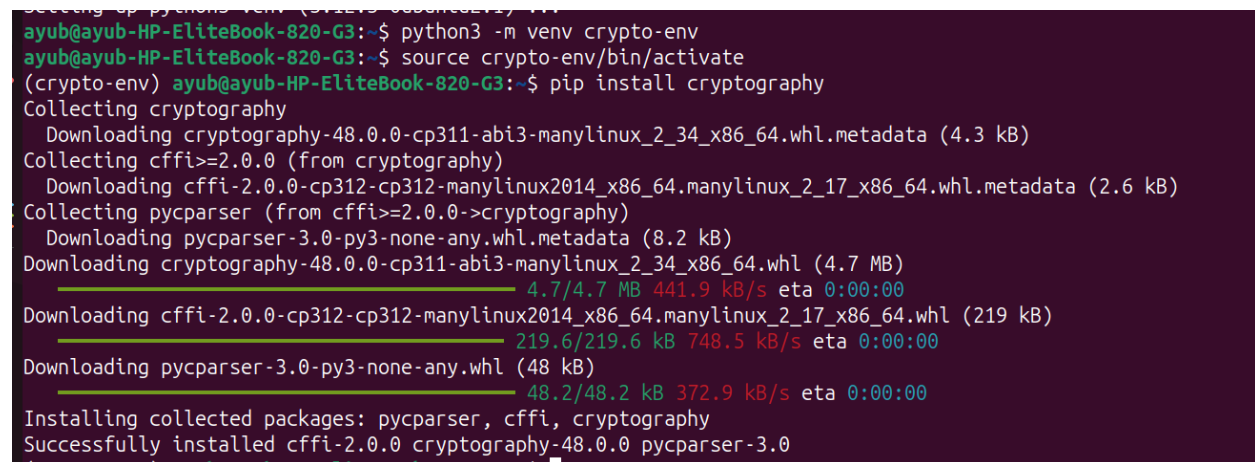
Week 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

A terminal window with a dark purple background. The prompt is 'ayub@ayub-HP-EliteBook-820-G3: ~'. The user enters 'python3 --version' and the output is 'Python 3.12.3'. Then the user enters 'code --version' and the output is '1.117.0', '10c8e557c8b9f9ed0a87f61f1c9a44bde731c409', and 'x64'. The prompt returns to 'ayub@ayub-HP-EliteBook-820-G3: ~\$'.

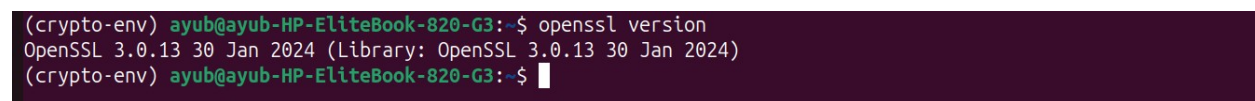
```
ayub@ayub-HP-EliteBook-820-G3: ~$ python3 --version
Python 3.12.3
ayub@ayub-HP-EliteBook-820-G3: ~$ code --version
1.117.0
10c8e557c8b9f9ed0a87f61f1c9a44bde731c409
x64
ayub@ayub-HP-EliteBook-820-G3: ~$
```

Figure 1: Installation of Python and VS Code

A terminal window with a dark purple background. The prompt is 'ayub@ayub-HP-EliteBook-820-G3: ~\$'. The user enters 'python3 -m venv crypto-env'. Then the user enters 'source crypto-env/bin/activate'. Then the user enters 'pip install cryptography'. The output shows the collection and downloading of cryptography-48.0.0, cffi-2.0.0, and pycparser-3.0. The prompt returns to '(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~\$'.

```
ayub@ayub-HP-EliteBook-820-G3: ~$ python3 -m venv crypto-env
ayub@ayub-HP-EliteBook-820-G3: ~$ source crypto-env/bin/activate
(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~$ pip install cryptography
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
  4.7/4.7 MB 441.9 kB/s eta 0:00:00
Downloading cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (219 kB)
  219.6/219.6 kB 748.5 kB/s eta 0:00:00
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
  48.2/48.2 kB 372.9 kB/s eta 0:00:00
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0
(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~$
```

Figure 2: Installation of Cryptography Libraries

A terminal window with a dark purple background. The prompt is '(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~\$'. The user enters 'openssl version'. The output is 'OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)'. The prompt returns to '(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~\$'.

```
(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~$
```

Figure 3: OpenSSL Installation Test

```
(crypto-env) ayub@ayub-HP-EliteBook-820-G3:~$ nano encrypt.py
(crypto-env) ayub@ayub-HP-EliteBook-820-G3:~$ python3 encrypt.py
Original: Hello World
Encrypted: qAAAAABqJWXYBFNZKrK0Xdqxf2kTk anuzCtj Njlicqa8ja8XP7rInS3v3Fy86V7z5VZaLinE018nVe5t0h3ZDKdnMfcd0K0q=
```

Figure 4: Local Encryption Script Execution

```
ayub@ayub-HP-EliteBook-820-G3:~$ mkdir CryptoProject
cd CryptoProject
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/ayub/CryptoProject/.git/
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ echo "# CryptoProject" > README.md
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git add .
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git commit -m "Initial commit"
[master (root-commit) d3b4b95] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git log --oneline
d3b4b95 (HEAD -> master) Initial commit
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$
```

Figure 5: GitHub Repository Initialization

Reflection

I learned how to set up a Python cryptography environment, configure OpenSSL and Git, and execute encryption scripts. The main challenge was resolving the pip installation restriction in Ubuntu. The screenshots demonstrate successful installation, configuration of security tools, encryption testing, and project repository initialization.

Class Activity 1: Manual Encryption

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

S- W

E- I

C- G

U- Y

R- V

I- M

T- X

Y- C

SECURITY using Caesar Cipher becomes- WIGYVMXC

Figure 6: Encrypt "SECURITY" using Caesar Cipher with shift 4.

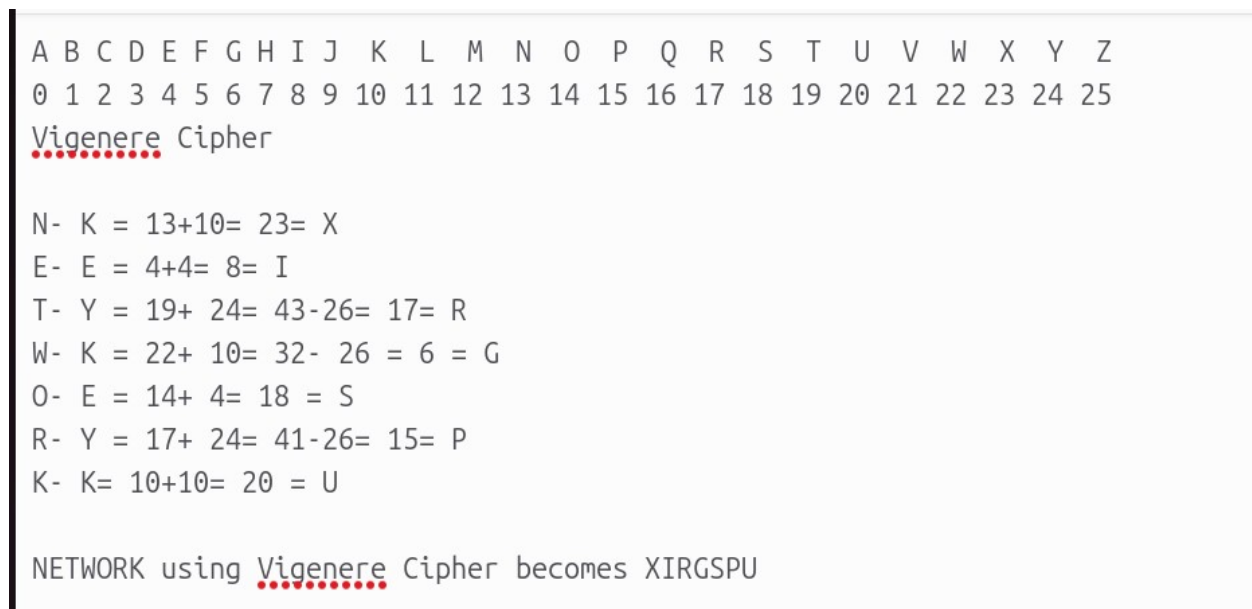


Figure 7: Encrypt “NETWORK” using Vigenère Cipher using keyword “KEY”.

Class Activity 2: Cipher Comparison Table

	Caesar Cipher	Vigenere Cipher
Encryption Method	Substitutes each letter by shifting it a fixed number of positions in the alphabet.	Multiple Caesar shifts based on a keyword.
Key Usage	A single numerical shift value ie. 4	A keyword that determines different shifts for different letters.
Security Level	Easy to crack using brute force or frequency analysis.	Higher security than Caesar Cipher as it uses multiple shifts.
Advantages	Simple to understand and implement	More secure than Caesar Cipher

	Caesar Cipher	Vigenere Cipher
	Requires little computation	Harder to break without knowing the keyword.
Weaknesses	Very easy to decrypt due to only 25 possible shifts.	Vulnerable to cryptanalysis if the keyword is short or repeated.
Ease Of Implementation	Very easy, can be done manually with minimal effort.	Moderately easy. Requires matching the keyword with the plaintext and performing multiple shifts

Practical Task Report: Comparison of Caesar Cipher and Vigenère Cipher

Introduction

Cryptography is the study of securing information by converting it into unreadable formats. Two basic classical encryption techniques are the Caesar Cipher and the Vigenère Cipher. Both are substitution ciphers used to understand basic encryption concepts.

Encryption Process

The Caesar Cipher works by shifting each letter in the plaintext by a fixed number of positions in the alphabet. For example, a shift of 4 changes A → E.

The Vigenère Cipher uses a keyword to determine different shift values for each letter. Each letter of the keyword defines a different Caesar shift, making the encryption more complex.

Advantages and Disadvantages

Caesar Cipher is simple and easy to implement but very weak in security.

Vigenère Cipher is more secure and harder to break, but it is more complex and still vulnerable if the keyword is short or reused.

Security Comparison

Caesar Cipher has very low security because it can be easily broken using brute force (only 25 possible shifts).

Vigenère Cipher is stronger because it uses multiple shifts, making frequency analysis more difficult, however it can still be cracked with modern cryptanalysis.

Modern Relevance

Caesar Cipher is mainly used for educational purposes only.

Vigenère Cipher is also mostly educational today, but it introduced ideas that influenced modern encryption techniques.

Conclusion

Both ciphers are important in learning cryptography basics. However, neither is secure for modern communication. Vigenère Cipher is stronger than Caesar Cipher but still not suitable for real-world security systems.

Extra Practical Task

The screenshot shows the 'Classical Cipher Studio' website. The header includes the site name '8gwifi.org' and navigation links: 'Tutorials', 'URL Shortener', 'JSON Beautifier', 'Temp Email', 'Hire Me!', and a 'Buy me a coffee' button. The main heading is 'Classical Cipher Studio' with a subtitle 'Type once – see many ciphers at a glance. Toggle Encode/Decode to switch direction.' The interface is divided into three main sections: 'Input & Mode', 'Parameters', and 'Outputs'. The 'Input & Mode' section has a 'Decode mode' checkbox and a text input field containing 'NETWORK'. The 'Parameters' section contains settings for 'Caesar Shift' (4), 'Affine A' (1), 'Affine B' (8), 'Vigenère Key' (KEY), 'Scytale Columns' (5), and 'Rail Fence Rails' (3). The 'Outputs' section displays the results for various ciphers: Caesar (shift 4) with output 'RIXASVO', ROT13 with 'ARGJBEX', ROT18 with 'ARGJBEX', ROT47 with '}t% (~#z', Affine (a=1, b=8) with 'VMBEWZS', Enigma (M3) with 'OZARWIY', Rail Fence (rails=3) with 'NOEWRTK', ROT5 (0-9) with 'NETWORK', Atbash with 'MVGDLIP', Vigenère (KEY) with 'XIRGSPU', and Scytale (cols=5) with 'NREKTWO'. Each output has a 'Copy' button.

Figure 8: Research and test one free online classical cipher simulator